Password-Typing Verification Across Sessions: Distance-Based Detectors vs. a One-Class SVM on the CMU Keystroke Benchmark

BİL462 Biometric Recognition · Project Phase 2 – Literature Review, System Design & Data Plan (topic A5) · Team: Bahadır İhsan Herdem (221101034), Ömer Efe Öztürk (221101059) · October 2026

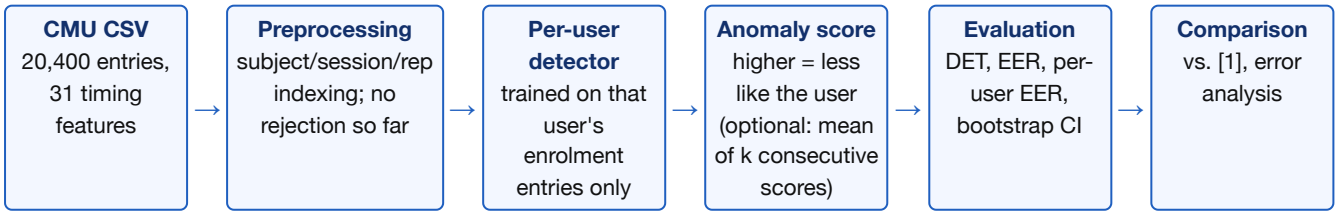
1. Problem statement and literature summary

Problem. A password can be stolen, but the way it is typed (key hold times and inter-key latencies) is harder to copy. We study *static* password-based keystroke verification: all subjects type the same fixed password, and a per-user detector decides whether a new entry was typed by the genuine user. Typing rhythm drifts from day to day, so results measured inside one session can be optimistic. Our question (topic A5): *how well do simple and learned detectors separate users who type the same password, and how much does performance degrade across sessions?* The biometric trait is keystroke dynamics (behavioural); the only dataset is the public CMU benchmark [1].

Literature. Keystroke timing as an identity signal goes back to Joyce & Gupta [2], who built a signature from the gaps between keystrokes (a Manhattan-type detector in the catalogue of [1]), and Monrose & Rubin [3], who studied it for authentication. Because published studies differed in data, password, training size and threshold rule, they could not be compared; Killourhy & Maxion [1] therefore released a benchmark (51 subjects × 8 sessions × 50 repetitions) and evaluated 14 anomaly detectors under one protocol. Their best detectors (scaled Manhattan, nearest-neighbour Mahalanobis, outlier count, one-class SVM) reach 9.6–10.2% mean EER, far from the error levels expected of access control, and detectors that scale features by their variability do best. A follow-up [6] examines which factors (algorithm, training amount, feature choice, updating, impostor practice, typist variability) drive error rates, which motivates our attention to evaluation conditions. One-class SVMs [5] were applied to keystroke novelty detection by Yu & Cho [4]. Recent work shifts toward learned embeddings: Siamese/triplet recurrent and Transformer models for free-text typing (TypeNet [8], TypeFormer [9]) and boosted/MLP/LSTM classifiers for fixed text [10, 11]; surveys are given in [7]. Two points matter for us. (i) A recent fixed-text paper [11] reports 94.75% accuracy and 1.0% EER on this dataset using 5- and 10-fold cross-validation, i.e. not a session-ordered split, so such numbers are not comparable to a verification protocol with temporal separation. (ii) Template ageing is now studied explicitly: a 2026 subject-disjoint study over eight weekly sessions finds that error rises with the enrolment-to-query gap for every matcher, including a scaled-Manhattan matcher, and that matcher choice affects accuracy more than ageing rate [12]; cross-device drift is addressed in [13]. These studies use other datasets, so the CMU benchmark with its fixed 8 sessions remains a clean place to measure drift for the simple-versus-learned comparison that topic A5 asks for.

Reference result. Killourhy & Maxion [1], Table 2, mean equal-error rate over subjects (training: first 200 repetitions; test: remaining 200 genuine repetitions vs. the first 5 repetitions of each of 50 impostors): scaled Manhattan **0.096**, Mahalanobis **0.110**, Manhattan **0.153**, one-class SVM **0.102**. We compare against these four values. *Preliminary check:* our own implementation of this protocol (41 evaluation subjects) gives 10.2%, 11.1%, 16.0%, and 11.6% respectively; on all 51 subjects the three distance detectors give 9.6%, 11.1%, 15.3%, matching [1].

2. System design



Features. The 31 values per entry for the password `.tie5Roanl`+Enter: 11 hold times (H), 10 down-down (DD) and 10 up-down (UD) latencies. **Detectors** (all written by us in numpy except the SVM kernel solver): *Manhattan* (L1 distance to the training mean), *scaled Manhattan* (each feature divided by its mean absolute deviation), *Mahalanobis* (inverse covariance, small ridge term because of few samples), and the **learned model: a one-class SVM** (RBF kernel, z-scored features, trained on the user's genuine entries only). Hyper-parameters of the SVM (ν, γ) are chosen on a separate tuning group of subjects (Section 3); the current choice is ν=0.05,

γ=0.03. The distance detectors have no tuned parameters. **Optional extensions:** (a) fusing the scores of k consecutive entries (already implemented, k∈{1,3,5,10}); (b) time permitting, a shared Siamese embedding network trained on subjects disjoint from the evaluated ones (cross-fitting over five subject groups), following the embedding approach of [8, 9]. A localhost login demo (enrolment sessions, then verification) reuses the same code; its data (team members only; instructor approval is requested before any collection) are never used in the evaluation.

3. Data, licence and splitting rule

Dataset. CMU Keystroke Dynamics Benchmark [1], `cs.cmu.edu/~keystroke`, file `DSL-StrongPasswordData.csv`: 51 subjects, 8 sessions (≥1 day apart), 50 repetitions each, 31 features. **Licence/terms:** the page names no formal licence; it states that the data and evaluation procedure are "freely available for use" as "a shared resource for public use", asks as a courtesy to be informed of publications based on the data, and asks users of a different methodology to make clear that it differs from the original paper. We will follow all three (our protocols C1/C2 and the drift study differ from [1] and are labelled as such; C3 replicates [1]). The data are not redistributed in our code archive; the README gives the download address. No other dataset and no own biometric data are used for the evaluation.

Splitting rule (topic A5). Every user has an individual model, so no person-disjoint split is needed for the distance detectors and the one-class SVM; instead data are split **by time**: training on earlier samples, testing on later ones, other subjects as impostors. Enrolment and test samples are always different. To tune the SVM without touching test people, the 51 subjects are split once (fixed seed) into a **tuning group of 10** subjects (ν, γ chosen on their sessions 1–3 for training and session 4 for validation) and an **evaluation group of 41** subjects; all reported results use only the 41. No impostor data are used in training, so impostor identities are never reused. A shared model (the optional embedding network) would be trained only on subjects not evaluated by it.

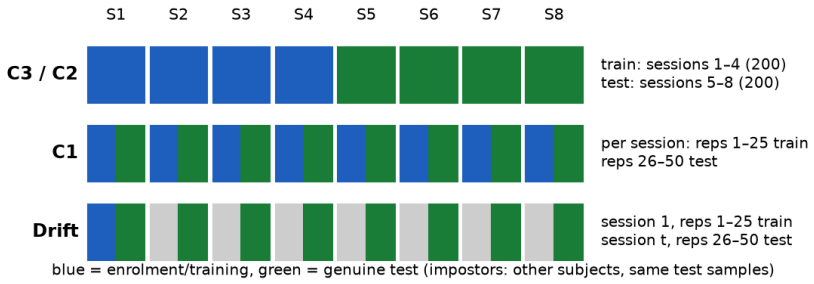


Figure 1. Training/test split per condition (S1–S8 = sessions).

Condition	Training (genuine user)	Test: genuine	Test: impostors	Purpose
C1 within-session	reps 1–25 of each session	reps 26–50, same session	other 40 subjects, same session, reps 26–50	optimistic baseline
C2 cross-session	sessions 1–4 (200)	sessions 5–8 (200)	other 40 subjects, sessions 5–8	realistic condition
Drift	session 1, reps 1–25	session t=1…8, reps 26–50	other 40 subjects, same session	controlled ageing curve (fixed training size)
C3 replication of [1]	first 200 reps	remaining 200	first 5 reps of each other subject	compare with reference

4. Number of comparisons, metrics and thresholds

Comparisons (41 evaluation subjects; each subject's detector scores the other 40 as impostors): C2 and C1 each give 41×200 = 8,200 genuine and 41×8,000 = 328,000 impostor comparisons; the drift study gives 25 genuine and 1,000 impostor comparisons per subject and test session; the replication C3 on all 51 subjects gives 10,200 genuine and 12,750 impostor comparisons. SVM tuning uses 10 subjects × 50 genuine and 9 × 50 impostor validation entries each. With 8,000 impostor scores per subject, FMR can be resolved down to about 0.1% (8 events), so we report no operating point below FMR = 0.1%; DET axes start at 0.1%.

Metrics. DET curve (main curve) and EER for each condition (within-session vs. cross-session; headline result = their difference), per-user EER distribution (mean hides consistent vs. erratic typists), and FNMR at FMR = 1%

and 0.1%. **Thresholds:** the EER is the standard threshold-free benchmark summary used in [1], computed per user on the test scores of that condition; it is not an operating point. For an operational point we additionally choose the threshold using *training sessions only* (leave-one-session-out genuine scores) and report the resulting FNMR/FMR on the test sessions. **Uncertainty:** 95% bootstrap confidence intervals obtained by resampling *subjects* (2,000 resamples), since entries of one subject are not independent (cf. subset resampling in [14]); differences between conditions use paired subject resampling. **Error analysis:** worst subjects/features, and the effect of training size, which is confounded in C1 vs. C2 (25 vs. 200 training entries) and is therefore separated by the drift experiment.

5. Tools, pretrained models and responsibility

No pretrained models are used. Python 3.14; numpy, scipy, pandas, matplotlib; scikit-learn [15] (`OneClassSVM` only for the SVM solver); Flask for the demo; PyTorch only if the optional embedding network is built. The three distance detectors, feature handling, evaluation conditions, metrics, bootstrap and figures are our own code; one script (`run_all.py`) reproduces every number. **Risks:** keystroke dynamics is a weak stand-alone factor ([1]); drift may partly reflect typing practice; browser timing differs from the benchmark's clock, so demo accuracy is not evidence; 41 subjects limit statistical power.

AI Use Declaration

Tools: Claude Code (Anthropic; model Claude Sonnet 5.5) and web search through it. **Used for:** writing and debugging the Python code (detectors, metrics, experiment scripts, figures), searching and summarising literature, drafting and editing this document. **How checked:** the implementation was validated by reproducing the published results of [1] (Manhattan 15.3%, scaled Manhattan 9.6%, Mahalanobis 11.1% vs. 15.3%, 9.6%, 11.0% reported); every reference was looked up online and its authors, title, venue and year were checked against the source page, and the team re-opened each link before submission; all design decisions (split rule, metrics, thresholds) were compared with the project description and topic A5 text. We can explain every part of the code and results.

References

Reference result and dataset (our results are compared against this study)

[1] K. S. Killourhy and R. A. Maxion, “Comparing Anomaly-Detection Algorithms for Keystroke Dynamics,” in *Proc. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, 2009, pp. 125–134. Dataset and evaluation script: https://www.cs.cmu.edu/~keystroke/

Foundational work: keystroke timing, detectors and the one-class SVM (reviewed)

[2] R. Joyce and G. Gupta, “Identity authentication based on keystroke latencies,” *Commun. ACM*, vol. 33, no. 2, pp. 168–176, 1990. https://doi.org/10.1145/75577.75582

[3] F. Monrose and A. D. Rubin, “Keystroke dynamics as a biometric for authentication,” *Future Generation Computer Systems*, vol. 16, pp. 351–359, 2000. https://www.passwordresearch.com/papers/paper60.html

[4] E. Yu and S. Cho, “Novelty detection approach for keystroke dynamics identity verification,” in *Intelligent Data Engineering and Automated Learning (IDEAL)*, 2003. https://dblp.org/rec/conf/ideal/YuC03.html

[5] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola and R. C. Williamson, “Estimating the support of a high-dimensional distribution,” *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001. https://doi.org/10.1162/089976601750264965

[6] K. Killourhy and R. Maxion, “Why did my detector do that?! Predicting keystroke-dynamics error rates,” in *Recent Advances in Intrusion Detection (RAID)*, LNCS 6307, 2010, pp. 256–276. https://www.cs.cmu.edu/~maxion/pubs/KillourhyMaxion10.pdf

Survey and recent work (reviewed, 2021–2026)

[7] R. Shadman, A. A. Wahab, M. Manno, M. Lukaszewski, D. Hou and F. Hussain, “Keystroke Dynamics: Concepts, Techniques, and Applications,” arXiv:2303.04605 (v3, 2025; accepted to ACM Computing Surveys). https://arxiv.org/abs/2303.04605

[8] A. Acien, A. Morales, J. V. Monaco, R. Vera-Rodriguez and J. Fierrez, “TypeNet: Deep Learning Keystroke Biometrics,” arXiv:2101.05570, 2021. https://arxiv.org/abs/2101.05570

[9] G. Stragapede, P. Delgado-Santos, R. Tolosana, R. Vera-Rodriguez, R. Guest and A. Morales, “TypeFormer: Transformers for Mobile Keystroke Biometrics,” arXiv:2212.13075, 2022 (rev. 2023). https://arxiv.org/abs/2212.13075

[10] H.-C. Chang, J. Li, C.-S. Wu and M. Stamp, “Machine Learning and Deep Learning for Fixed-Text Keystroke Dynamics,” arXiv:2107.00507, 2021. https://arxiv.org/abs/2107.00507

[11] K. Sasikumar and S. Nagarajan, “LSTM-based ensemble models for keystroke dynamics authentication: integrating explainable AI for transparency,” *Frontiers in Artificial Intelligence*, vol. 9, 2026. https://doi.org/10.3389/frai.2026.1860248

[12] S. Parkinson, S. Khan, N. Liu and Q. Xu, “Template Ageing and Longitudinal Verification in Fixed-Text Keystroke Dynamics: A Subject-Disjoint Study Across Eight Weeks,” arXiv:2609.29851, 2026. https://arxiv.org/abs/2609.29851

[13] N. Kaluarachchi, S. Kandanaarachchi, K. Moore, A. Arakala and C. Sanderson, “Transfer Learning of Keystroke Dynamics for Cross-Device User Authentication,” arXiv:2608.16334, 2026. https://arxiv.org/abs/2608.16334

Evaluation methodology and tools

[14] R. M. Bolle, N. K. Ratha and S. Pankanti, “Error analysis of pattern recognition systems—the subsets bootstrap,” *Computer Vision and Image Understanding*, vol. 93, no. 1, pp. 1–33, 2004. https://doi.org/10.1016/j.cviu.2003.08.002

[15] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html